

Master's thesis
Master's Programme in Computer Science

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ha Vien

August 7, 2026

Supervisor(s): Keijo Heljanko, Christoph Luneburg, Petteri Kotimaa

Examiner(s):

University of Helsinki
Faculty of Science

P. O. Box 68 (Pietari Kalmin katu 5)
00014 University of Helsinki

HELSINGIN YLIOPISTO — HELSINGFORS UNIVERSITET — UNIVERSITY OF HELSINKI

Tiedekunta — Fakultet — Faculty	Koulutusohjelma — Utbildningsprogram — Degree programme
Faculty of Science	Master's Programme in Computer Science

Tekijä — Författare — Author

Ha Vien

Työn nimi — Arbetets titel — Title

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Työn laji — Arbetets art — Level	Aika — Datum — Month and year	Sivumäärä — Sidantal — Number of pages
Master's thesis	August 7, 2026	27

Tiivistelmä — Referat — Abstract

Abstract to be written last.

ACM Computing Classification System (CCS):
Software and its engineering → Software notations and tools → General programming languages
Computer systems organization → Real-time systems → Real-time system specification
Networks → Network types → Mobile networks

Avainsanat — Nyckelord — Keywords

Rust, C++, RAN control plane, tail latency, memory safety, fault injection, FFI

Säilytyspaikka — Förvaringsställe — Where deposited

Helsinki University Library

Muita tietoja — Övriga uppgifter — Additional information

Software study track

Contents

1 Introduction **1**

2 Background: Systems Languages for Latency-Critical Software **3**

3 Problem Context: UE Context Management and Predictability **5**

4 Objectives **7**

5 Research Methodology **9**

6 Design and Implementation **11**

7 Experimental Setup and Evaluation **13**

8 Discussion **15**

9 Conclusion and Future Work **17**

Bibliography **19**

Appendix A Fault-injection example **27**

1. Introduction

- The 5G RAN control plane (gNB-CU-CP) is being softwarized onto commodity servers [1, 2, 48]: it is large, stateful, highly concurrent, and bound by millisecond-scale procedure budgets [5], so a defect or a tail-latency regression there degrades the whole site [18, 12].
- That software is written predominantly in C++, which supplies the required control and performance but leaves memory-safety and concurrency defects to the programmer [56, 41]; Rust promises the same performance with those defects rejected at compile time [33, 29, 30].
- The problem: telecom vendors face growing pressure to adopt memory-safe languages [17, 45, 52] but have no direct, controlled evidence on predictability, prevented-defect scope, or migration cost for a representative control-plane subsystem [31, 8].
- Four research questions frame the work: RQ1 tail latency and jitter under overload [18], RQ2 which defect classes are eliminated at compile time [61], RQ3 the overhead of Rust's safety model [64], and RQ4 the cost of incremental adoption inside a C++ codebase [35, 23].
- Contributions: a reproducible benchmarking framework [25], a frozen specification with two parity-tested implementations, a defect-class taxonomy including the boundary of the guarantee, and evidence-based migration guidelines.

2. Background: Systems Languages for Latency-Critical Software

- The NG-RAN architecture splits the gNB into central and distributed units and the central unit again into control and user planes; UE Context Management sits inside the gNB-CU-CP, connected over the standardised F1, E1, and NG interfaces [1, 2, 3, 4].
- Predictability rather than average speed decides whether deadlines are met, so the governing metrics are the p99/p99.9 tail and jitter [18, 12, 27], measured past saturation [36] and captured without coordinated omission using high-dynamic-range histograms [58].
- C++ offers zero-overhead abstractions, RAII, and no garbage collector, but safety remains the programmer's responsibility [56] and sanitizers detect only what a given test happens to execute [44].
- Rust enforces memory safety and data-race freedom at compile time through ownership, borrowing, and the `Send`/`Sync` traits [33, 30], with machine-checked foundations [29, 28], `unsafe` as an explicitly delimited escape hatch [9, 61], and no garbage collector.
- The comparison is therefore C++ against Rust (managed languages are excluded on tail-latency grounds [47]), with the concurrency model held fixed and C-ABI interoperability as the adoption path [53, 32]; Rust's systems credentials are established elsewhere [6, 34, 42], but no prior study isolates the language this way [31, 8].

3. Problem Context: UE Context Management and Predictability

- UE Context Management owns the per-UE context lifecycle: a four-state machine (Idle, Connected, Reconfiguring, Released) [4, 1] driven by a small, lifecycle-oriented operation set in which every illegal operation returns a typed error.
- It is chosen because it is both representative (per-UE mutable state, strict lifecycle, high concurrency, hard latency targets [5]) and tractable enough to specify precisely and parity-test in two languages; control-plane latency is an active target in the adjacent core network [49, 39, 46].
- Five defect classes lie inside Rust's guarantee [30] and each has a natural incarnation here: use-after-free, double-free, iterator invalidation, data race, and unguarded shared mutation [56, 51].
- Two further classes are carried deliberately from outside the guarantee, a lock-order deadlock [63, 57] and a lost update [50], so that the study locates the boundary instead of confirming a foregone conclusion.
- Fan-out and fleet scale turn the p99.9 of a single operation into a sustained rate of degraded procedures [18], which is why the load sweep is pushed over the overload cliff [36].

4. Objectives

- A stated literature-search protocol returns no study measuring the tail latency of a Rust and a C++ implementation of the same stateful control-plane subsystem under overload; the closest work compares languages for RAN applications [31] and async frameworks for 5G control-plane procedures [8].
- Five objectives map onto the research questions: quantify predictability (O1) [18], determine prevented defect classes (O2) [61], measure resource overhead (O3) [64], build and measure a migration prototype (O4) [38], and deliver reusable artifacts and guidance (O5).
- Three hypotheses are committed to in advance: H1 performance equivalence below saturation [19, 11], H2 compile-time prevention of the defect classes C++ leaves to best-effort detection [30, 56], and H3 negligible interoperability cost [53].
- Success means decision-grade evidence: repeats with confidence intervals [25], defects demonstrated on compilable programs, migration cost read off a working prototype, and reproducibility from published artifacts.
- Scope is drawn to protect internal validity; the full gNB, PHY/L2 and the user plane, captured traffic traces, managed languages [47], and production operations are explicitly out.

5. Research Methodology

- The approach is empirical and constructive: rather than comparing two pre-existing systems, the study builds a matched pair of implementations from one frozen specification so that language is the only systematic difference [37, 43].
- RQ1 and RQ3 are answered by an open-loop load sweep from light load into overload, with latency captured in identically configured HDR histograms [58] and resource use sampled through the operating system.
- RQ2 is answered by controlled fault injection [44]: a fixed catalogue is confronted in both languages, with recorded expectations in each direction so an unexpected outcome surfaces as a discrepancy.
- RQ4 is answered by a working C++-to-Rust FFI prototype measuring seam size, the scope of `unsafe` [9], and per-call crossing cost [38, 14].
- Fairness is the central experimental control (frozen specification, matched allocator and optimisation, forced-idiom log), and results are reported as medians with bootstrap confidence intervals plus exact paired permutation tests under Holm correction [25]; all data is synthetic, so no data-protection concerns arise.

6. Design and Implementation

- A language-agnostic specification is frozen before any code is written, fixing the public API, state machine, data structures, algorithmic strategy, and sharding and concurrency model as the fairness baseline [37, 43].
- Both versions use one shared-nothing, thread-per-shard architecture [21]: eight shards, bounded blocking MPSC queues, single-use completion handles, strict FIFO per shard, and a single atomic counter as the only shared mutable state.
- The run-time substrate is equalised: the same jemalloc allocator (with a system-allocator control), identically configured HDR histograms [58], matched `-O2` and `opt-level = 2`, and pinned shard-selection and intra-shard hash functions.
- Where the languages must diverge, the forced-idiom log records it: C++ hand-writes the bounded queue Rust gets from `sync_channel`, and C++'s tagged request struct plus response variant is larger and more branch-heavy than Rust's per-variant typed sender, leaving Rust longer in lines but lower in total cyclomatic complexity [62, 7].
- The Rust core contains no `unsafe` at all [9, 22], and functional equivalence is tested rather than assumed by diffing canonical transcripts from a language-neutral conformance suite.

7. Experimental Setup and Evaluation

- Two environments are distinguished: everything reported was measured on a WSL2 development host, which is adequate for parity, fault injection, and the FFI structure but not for tail-latency claims [36]; the bare-metal protocol is specified and scripted but not yet run.
- The workload is a deterministic, spec-justified RAN lifecycle generator [4] driven on a fixed open-loop send schedule [58] and swept from nominal load through saturation, with the four language-by-allocator variants interleaved within every repeat [25].
- RQ1 and RQ3 performance: the four variants are indistinguishable at the median and equal in throughput and CPU [19, 11], while the tail is not resolvable here because the confidence interval on one variant's p99 exceeds the spread between all four.
- RQ3 memory is the one cleanly resolved performance result: the Rust build carries a few MiB more peak resident set under either allocator, with the same sign in every repeat and a larger effect than the allocator's own [64, 16].
- RQ2 and RQ4: all five in-scope defect classes are compile errors in safe Rust [30] while each compiles in C++ and needs the right sanitiser [56], the two boundary classes compile in both [63], and the twelve-entry-point FFI seam keeps the core `unsafe`-free at a sub-nanosecond per-call cost [38].

8. Discussion

- The findings are reported with an evidence status per research question: safety and migration are final because they do not depend on the measurement environment, while performance and resource results are preliminary [25].
- On performance the honest reading is that the median question is settled and the tail question is open [18], which keeps the eventual bare-metal result a finding rather than a confirmation; the memory penalty is bounded but unexplained without hardware counters [64, 26].
- On safety the difference is one of kind rather than degree, and it has a definite edge: Rust eliminates data races, not race conditions, and aliasing errors, not lock-ordering errors [30, 50, 63]; ecosystem studies place the residual risk in `unsafe` code and dependencies [10, 65].
- On migration, an `unsafe`-free core is a statement about what the compiler checks and not about what the C++ host sharing its address space can corrupt [53, 32], so the gain is that the component stops contributing new defects [59].
- The hypotheses are revisited (H1 half-answered, H2 supported precisely and refuted in its loose reading, H3 supported), engineering guidelines are derived, and related work [13, 60, 40], threats to validity, and the professional obligation not to over-claim are addressed [24].

9. Conclusion and Future Work

- The thesis compared modern C++ and Rust on one UE Context Management service built from a single frozen specification, isolating language effects from design and skill differences along four dimensions [37].
- Safety: safe Rust rejects all five injected memory-safety and data-race classes at compile time [29, 30], while the two boundary classes compile and misbehave exactly as they do in C++ [50].
- Migration: an unmodified Rust core can be driven from C++ across a thin, `unsafe`-isolated seam at sub-nanosecond per-call cost [38, 54], so the cost is API design and the learning curve [24, 15].
- Performance: the languages are equivalent at the median and in CPU and throughput [19], the Rust build carries a few MiB more peak RSS [64], and the tail is not yet measurable on a shared host [36].
- Future work runs the specified bare-metal campaign first, then extends to production traces, further subsystems [20, 55], hardware counters, async runtimes [8], in-process heap isolation [32], and a longitudinal migration case study [35, 23].

Bibliography

[1] 3GPP. NG-RAN; architecture description. Technical Specification (TS) 38.401, 3rd Generation Partnership Project (3GPP), 2022. V17.5.0, Release 17.

[2] 3GPP. NR; NR and NG-RAN overall description; stage 2. Technical Specification (TS) 38.300, 3rd Generation Partnership Project (3GPP), 2022. V17.5.0, Release 17.

[3] 3GPP. NG-RAN; E1 application protocol (E1AP). Technical Specification (TS) 38.463, 3rd Generation Partnership Project (3GPP), 2023. V17.5.0, Release 17.

[4] 3GPP. NR; radio resource control (RRC); protocol specification. Technical Specification (TS) 38.331, 3rd Generation Partnership Project (3GPP), 2023. V17.5.0, Release 17.

[5] 3GPP. Service requirements for the 5G system; stage 1. Technical Specification (TS) 22.261, 3rd Generation Partnership Project (3GPP), 2023. V18.6.0, Release 18.

[6] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa. Firecracker: Lightweight virtualization for serverless applications. In *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 419–434. USENIX Association, 2020.

[7] L. Ardito, L. Barbato, R. Coppola, and M. Valsesia. Evaluation of Rust code verbosity, understandability and complexity. *PeerJ Computer Science*, 7:e406, 2021.

[8] A. Arvidsson and M. Kaller. Evaluating asynchronous programming frameworks for 5G control plane procedures. Master's thesis, Linköping University, Department of Computer and Information Science, 2025. Carried out at Ericsson AB.

[9] V. Astrauskas, C. Matheja, F. Poli, P. Müller, and A. J. Summers. How do programmers use unsafe Rust? *Proceedings of the ACM on Programming Languages*, 4(OOPSLA):136:1–136:27, 2020.

[10] Y. Bae, Y. Kim, A. Askar, J. Lim, and T. Kim. Rudra: Finding memory safety bugs in Rust at the ecosystem scale. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*, pages 84–99, 2021.

[11] A. Balasubramanian, M. S. Baranowski, A. Burtsev, A. Panda, Z. Rakamarić, and L. Ryzhyk. System programming in Rust: Beyond safety. In *Proceedings of the 16th Workshop on Hot Topics in Operating Systems (HotOS)*, pages 156–161, 2017.

[12] L. Barroso, M. Marty, D. Patterson, and P. Ranganathan. Attack of the killer microseconds. *Communications of the ACM*, 60(4):48–54, 2017.

[13] W. Bugden and A. Alahmar. Rust: The programming language for safety and performance. *arXiv preprint arXiv:2206.05503*, 2022.

[14] X. Cai, J. Liu, C. Liu, L. Bao, Y. Yu, and L. Jiang. Fortifying the seams between C/C++ and Rust: Characterizing bugs in interop tools. *ACM Transactions on Software Engineering and Methodology*, 2026.

[15] W. Crichton and S. Krishnamurthi. Profiling programming language learning. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA1):29–54, 2024.

[16] C. Cui and H. Xu. Unleashing the efficiency of Rust: An empirical study of performance bugs in Rust projects. In *Proceedings of the 2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE)*, pages 371–381, 2025.

[17] Cybersecurity and Infrastructure Security Agency and National Security Agency and Federal Bureau of Investigation and others. The case for memory safe roadmaps: Why both C-suite executives and technical experts need to take memory safe coding seriously. Cybersecurity information sheet, CISA, NSA, FBI, ASD's ACSC, CCCS, NCSC-UK, NCSC-NZ, CERT NZ, Dec. 2023.

[18] J. Dean and L. A. Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013.

[19] P. Emmerich, S. Ellmann, F. Bonk, A. Egger, E. García Sánchez-Torija, T. Günzel, S. Di Luzio, A. Obada, M. Stadlmeier, S. Voit, and G. Carle. The case for writing network drivers in high-level programming languages. In *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*, 2019.

[20] M. Emre, P. Boyland, A. Parekh, R. Schroeder, K. Dewey, and B. Hardekopf. Aliasing limits on translating C to safe Rust. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1):551–579, 2023.

[21] P. Enberg, A. Rao, and S. Tarkoma. The impact of thread-per-core architecture on application tail latency. In *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*, pages 1–8, 2019.

[22] A. N. Evans, B. Campbell, and M. L. Soffa. Is Rust used safely by software developers? In *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*, pages 246–257, 2020.

[23] Y. Fan, C. Wang, Z. Dong, T. Liu, and H. Wang. An empirical analysis of Rust integration in the Android open source project. In *Companion Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering (FSE Companion)*, pages 545–556, 2026.

[24] K. R. Fulton, A. Chan, D. Votipka, M. Hicks, and M. L. Mazurek. Benefits and drawbacks of adopting a secure programming language: Rust as a case study. In *Proceedings of the 17th Symposium on Usable Privacy and Security (SOUPS)*, pages 597–616. USENIX Association, 2021.

[25] A. Georges, D. Buytaert, and L. Eeckhout. Statistically rigorous Java performance evaluation. In *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*, pages 57–76, 2007.

[26] S. Harris, S. Cooksey, M. Vollmer, and M. Batty. Rust for Morello: Always-on memory safety, even in unsafe code (experience paper). In *Proceedings of the 37th European Conference on Object-Oriented Programming (ECOOP)*, volume 263 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 39:1–39:27, 2023.

[27] J. Huang, B. Mozafari, G. Schoenebeck, and T. F. Wenisch. A top-down approach to achieving performance predictability in database systems. In *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD)*, pages 745–758, 2017.

[28] R. Jung, H.-H. Dang, J. Kang, and D. Dreyer. Stacked borrows: An aliasing model for Rust. *Proceedings of the ACM on Programming Languages*, 4(POPL):41:1–41:32, 2020.

[29] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. RustBelt: Securing the foundations of the Rust programming language. *Proceedings of the ACM on Programming Languages*, 2(POPL):66:1–66:34, 2018.

[30] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. Safe systems programming in Rust. *Communications of the ACM*, 64(4):144–152, 2021.

[31] R. Karlbäck and A. Orö. Holistic view on alternative programming languages for radio access network applications in cloud and embedded deployments. Master's thesis, Linköping University, Department of Computer and Information Science, 2021. Carried out at Ericsson AB.

[32] P. Kirth, M. Dickerson, S. Crane, P. Larsen, A. Dabrowski, D. Gens, Y. Na, S. Volckaert, and M. Franz. PKRU-Safe: Automatically locking down the heap between safe and unsafe languages. In *Proceedings of the 17th European Conference on Computer Systems (EuroSys)*, pages 132–148, 2022.

[33] S. Klabnik and C. Nichols. *The Rust Programming Language*. No Starch Press, San Francisco, CA, 2nd edition, 2023.

[34] A. Levy, B. Campbell, B. Ghena, P. Pannuto, P. Dutta, and P. Levis. The case for writing a kernel in Rust. In *Proceedings of the 8th Asia-Pacific Workshop on Systems (APSys)*, pages 1–7, 2017.

[35] H. Li, L. Guo, Y. Yang, S. Wang, and M. Xu. An empirical study of Rust-for-Linux: The success, dissatisfaction, and compromise. In *Proceedings of the 2024 USENIX Annual Technical Conference (USENIX ATC)*, pages 425–443. USENIX Association, 2024.

[36] J. Li, N. K. Sharma, D. R. K. Ports, and S. D. Gribble. Tales of the tail: Hardware, OS, and application-level sources of tail latency. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, pages 9:1–9:14, 2014.

[37] Z. Li, T. Huang, V. Narayanan, and A. Burtsev. Understanding the overheads of hardware and language-based IPC mechanisms. In *Proceedings of the 11th Workshop on Programming Languages and Operating Systems (PLOS)*, pages 53–61, 2021.

[38] Z. Li, J. Wang, M. Sun, and J. C. S. Lui. Detecting cross-language memory management issues in Rust. In *Computer Security – ESORICS 2022*, pages 680–700, 2022.

[39] Y.-S. Liu, S. Qi, P.-Y. Lin, H.-S. Tsai, K. K. Ramakrishnan, and J.-C. Chen. L25GC+: An improved, 3GPP-compliant 5G core for low-latency control plane operations. In *Proceedings of the 2023 IEEE 12th International Conference on Cloud Networking (CloudNet)*, pages 203–211, 2023.

[40] I. McCormack, T. Dougan, S. Estep, H. Hibshi, J. Aldrich, and J. Sunshine. A mixed methods study on the implications of unsafe Rust for interoperation, encapsulation, and tooling. *ACM Transactions on Software Engineering and Methodology*, 2026.

[41] M. Miller. Trends, challenges, and strategic shifts in the software vulnerability mitigation landscape. BlueHat IL, Microsoft Security Response Center (MSRC), 2019. Approximately 70% of Microsoft CVEs are memory-safety issues.

[42] C. Mo, L. Wang, S. Li, K. Hu, and B. Jiang. Rust-Shyper: A reliable embedded hypervisor supporting VM migration and hypervisor live-update. *Journal of Systems Architecture*, 142:102948, 2023.

[43] V. Narayanan, T. Huang, D. Detweiler, D. Appel, Z. Li, G. Zellweger, and A. Burtsev. RedLeaf: Isolation and communication in a safe operating system. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 21–39. USENIX Association, 2020.

[44] R. Natella, D. Cotroneo, and H. S. Madeira. Assessing dependability with software fault injection: A survey. *ACM Computing Surveys*, 48(3):44:1–44:55, 2016.

[45] Office of the National Cyber Director. Back to the building blocks: A path toward secure and measurable software. Technical report, The White House, Office of the National Cyber Director (ONCD), Washington, DC, Feb. 2024.

[46] C. Park and J. Nam. CtrlBench: Towards accurate API-level benchmarking in 5G core control plane. In *Proceedings of the 2025 16th International Conference on Information and Communication Technology Convergence (ICTC)*, pages 1832–1837, 2025.

[47] R. Pereira, M. Couto, F. Ribeiro, R. Rua, J. Cunha, J. P. Fernandes, and J. Saraiva. Energy efficiency across programming languages: How do energy, time, and memory relate? In *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering (SLE)*, pages 256–267, 2017.

[48] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia. Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges. *IEEE Communications Surveys & Tutorials*, 25(2):1376–1411, 2023.

[49] Z. A. Qazi, M. Walls, A. Panda, V. Sekar, S. Ratnasamy, and S. Shenker. A high performance packet core for next generation cellular networks. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication (SIGCOMM)*, pages 348–361, 2017.

[50] B. Qin, Y. Chen, H. Liu, H. Zhang, Q. Wen, L. Song, and Y. Zhang. Understanding and detecting real-world safety issues in Rust. *IEEE Transactions on Software Engineering*, 50(6):1306–1324, 2024.

[51] B. Qin, Y. Chen, Z. Yu, L. Song, and Y. Zhang. Understanding memory and thread safety practices and issues in real-world Rust programs. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 763–779, 2020.

[52] A. Rebert and C. Kern. Secure by design: Google's perspective on memory safety. Google Security Engineering, Technical Report, 2024.

[53] E. Rivera, S. Mergendahl, H. Shrobe, H. Okhravi, and N. Burow. Keeping safe Rust safe with Galeed. In *Proceedings of the 37th Annual Computer Security Applications Conference (ACSAC)*, pages 824–836, 2021.

[54] L. Schuermann, A. Thomas, and A. Levy. Encapsulated functions: Fortifying Rust's FFI in embedded systems. In *Proceedings of the 1st Workshop on Kernel Isolation, Safety and Verification (KISV)*, pages 41–48, 2023.

[55] A. Sharma, S. Sharma, S. R. Tanksalkar, S. Torres-Arias, and A. Machiry. Rust for embedded systems: Current state and open problems. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2296–2310, 2024.

[56] L. Szekeres, M. Payer, T. Wei, and D. Song. SoK: Eternal war in memory. In *Proceedings of the 2013 IEEE Symposium on Security and Privacy (S&P)*, pages 48–62, 2013.

[57] R. Tang, M. Wang, X. Sun, L. Huang, Y. Huang, and X. Ma. Converos: Practical model checking for verifying Rust OS kernel concurrency. In *Proceedings of the 2025 USENIX Annual Technical Conference (USENIX ATC)*, pages 1143–1159. USENIX Association, 2025.

[58] G. Tene. HdrHistogram: A high dynamic range histogram, and the coordinated omission problem. http://hdrhistogram.org/, 2015. Accessed 2026.

[59] J. Vander Stoep and S. Hines. Memory safe languages in Android 13. Google Security Blog, 2022.

[60] Y. Xiao, Y. Chen, X. Zhao, J. Duan, X. Zhang, W. Li, and X. Tao. rpkt: A generic, safe, and efficient userspace packet processing library in Rust. In *Network and Parallel Computing (NPC)*, pages 454–465, 2024.

[61] H. Xu, Z. Chen, M. Sun, Y. Zhou, and M. R. Lyu. Memory-safety challenge considered solved? an in-depth study with all Rust CVEs. *ACM Transactions on Software Engineering and Methodology*, 31(1):3:1–3:25, 2021.

[62] T. Yin. Lizard: An extensible cyclomatic complexity analyzer for many programming languages. http://www.lizard.ws/, 2025. Version 1.23.0. Accessed 2026.

[63] Y. Zhang, K. Zhang, and G. Liu. Static deadlock detection for Rust programs. arXiv preprint arXiv:2401.01114, 2024.

[64] Y. Zhang, Y. Zhang, G. Portokalidis, and J. Xu. Towards understanding the runtime performance of Rust. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 140:1–140:6, 2022.

[65] X. Zheng, Z. Wan, Y. Zhang, R. Chang, and D. Lo. A closer look at the security risks in the Rust ecosystem. *ACM Transactions on Software Engineering and Methodology*, 33(2):34:1–34:30, 2024.

Appendix A. Fault-injection example

- Illustrates one case from the RQ2 catalogue, the use-after-free, in both languages side by side [56].
- In C++ an alias into the context map survives the `erase` that frees it, and the program compiles cleanly.
- It is flagged only at runtime, and only by AddressSanitizer on a path a test actually exercises [44].
- The analogous safe-Rust program does not compile: the borrow checker forbids mutating the map while a reference into it is live (`E0499`) [33, 28].
- The pair makes the kind-of-guarantee difference concrete rather than argued from language semantics [30].